

but tell me exactly how the semantic engine is going to establish these facts and relationships; how will it work

The important thing is that the semantic engine should not start by trying to "understand the repository" in the LLM sense. It should build a typed, incrementally maintained model of observable facts from several authoritative sources, then derive relationships through deterministic analyses. The agent comes later and reasons over that model.

I would make the engine essentially a compiler/indexer for software projects. Its input is Git plus the forge plus the checked-out source tree plus build/test metadata. Its output is an append-only-ish repository model containing entities, events, relationships, and provenance. Every fact should answer "where did this come from?" and "at what repository/forge state was this true?"

At the bottom is an ingestion layer. For Git, use libgit2 or the git CLI initially, with Rust's git2 ecosystem being a practical choice. The engine walks commits, trees, parents, branches, tags, refs, renames, authors, timestamps, commit messages, and patches. It stores identities based on immutable Git object IDs rather than filenames or URLs. A commit becomes an entity; each parent relationship is explicit; each changed path becomes a change event; each added/deleted/modified line range can be represented as a change record. This immediately gives you a deterministic temporal graph of the repository.

Then you parse the source. This is where the compiler-like part becomes valuable. Use Tree-sitter where broad language coverage matters, and language-specific parsers/compiler APIs where deeper semantic information is available. You don't merely store ASTs. You extract normalized entities: modules, namespaces, types, functions, methods, constants, macros, interfaces, imports, calls, definitions, references, tests, configuration entries, and so forth. Each entity gets a stable identity as far as the language permits. A function isn't merely "function named `foo`"; it becomes something resembling "function `foo` in module X at repository revision Y." You then establish definitions, references, calls, containment, imports, inheritance/implementation, and test relationships.

The really important trick is to maintain mappings between Git changes and these semantic entities. A commit says that lines 120–170 of file X changed. The parser says that those lines belong to function Y. Therefore the engine can deterministically derive "commit C modified function Y." If the function was renamed or moved, Git's rename information and semantic similarity can be used to maintain continuity. With enough machinery, you can eventually say things like "this function has been modified by 17 commits across three architectural eras," which is considerably more meaningful than "this file has 43 commits."

Then comes the forge ingestion layer. GitHub and Codeberg objects get normalized into forge-neutral entities: repository, issue, pull request, comment, review, user, label, milestone,

release, etc. The engine records identifiers and timestamps from the forge and maintains provenance back to the original object. An issue's textual content is not itself a semantic fact about the code. It is an artifact from which deterministic signals can be extracted and from which an agent can later infer hypotheses.

There are several levels of relationship extraction. The first is explicit relationships. GitHub tells you that PR 123 closes issue 456. A commit belongs to a branch. A PR contains commits. A review belongs to a PR. An issue has comments. These should simply become edges in the model with essentially zero ambiguity.

The next level is Git-derived relationships. A commit modifies a file. A file contains a function. Therefore the commit modifies the function. A test calls function X. Therefore the test covers X, at least structurally. Commit A is an ancestor of commit B. Therefore B occurred after A. Commit B reverts A. Therefore B reverses A. A PR contains commits C, D, and E. Therefore its patch affects the union of the semantic entities touched by those commits. These are graph joins, not AI judgments.

Then you can do historical analysis. Suppose function `parse_config` has been modified by commits A, B, F, G, and J. The engine can calculate churn, modification frequency, authorship distribution, periods of instability, revert frequency, coupling with other symbols, and which tests tend to change alongside it. If issues 14, 31, and 72 all have forge text referring to configuration parsing and their associated PRs modify that function, you now have a deterministic cluster candidate. You don't yet say "these issues are the same bug." You say "these issues have overlapping textual and structural evidence."

This distinction is essential. I would give every relationship a type and evidence set. For example, `Issue -> relates_to -> Function` might have evidence `{PR 31 modified function, PR 72 modified function, issue discussion references config parser}`. A separate relationship could be `Issue -> likely_related_to -> Issue`, whose evidence is similarity plus shared implementation history. The second is an inference candidate, not an established fact.

Textual analysis can still be deterministic and surprisingly useful. You can tokenize issue titles/descriptions, comments, commit messages, PR descriptions, and documentation. Normalize identifiers, extract code symbols, detect issue/PR references, recognize file paths, and perform lexical similarity. You can use BM25, TF-IDF, n-gram similarity, or embeddings later. You don't need an LLM to discover that "completion timeout" and "completion hangs indefinitely" are lexically similar. The engine can produce a ranked candidate relationship with its evidence.

Embeddings are where I would be somewhat conservative. They can be an excellent retrieval mechanism, but I would never make "vector similarity = semantic truth." Put embeddings beside the deterministic evidence. If an issue's text is similar to another issue and both touch the same symbols through historical PRs, that is much stronger than either signal alone. The semantic engine can calculate a composite evidence score while retaining every component: lexical similarity, symbol overlap, file overlap, temporal proximity, shared PRs, shared tests, etc.

The same approach works for contributors. Git gives you authorship of commits; the forge gives you issue comments, PRs, reviews, and discussions. You can construct deterministic activity profiles: contributor X has modified these modules, reviewed these areas, opened these issues, and historically participated in these architectural discussions. You can then derive relationships such as "contributor X has repeatedly worked on subsystem Y." That is factual. An agent can subsequently infer "X appears to be particularly knowledgeable about Y," but the latter remains an interpretation.

Tests deserve their own analysis because they provide an unusually powerful bridge between behavior and implementation. The engine can statically associate tests with symbols and files, and dynamically associate them through test execution where possible. A test failure can then enter the same event graph: test T failed at commit C; T exercises symbols X and Y; commit C modified X; issue Z describes a behavior matching T. That gives you a deterministic chain connecting an activity event to potentially relevant code.

The build system can contribute another layer. Cargo, dune, npm, CMake, Meson, Go modules, etc. can expose package/dependency relationships. The engine can normalize those into a dependency graph. Then an apparently local patch can be analyzed for downstream impact. "This function changed" becomes "this function is used by these 47 symbols across these six packages, and three tests exercise the affected behavior." Again, the agent isn't discovering this from prose; it is receiving a mechanically established impact surface.

Now consider an actual investigation. An issue arrives saying "completion stops working after reopening a project." The engine creates an event node for the issue. It extracts explicit references and code identifiers. It searches lexical evidence. It finds PRs historically associated with the same subsystem. It resolves those PRs to commits. It maps commits to changed functions. It maps those functions to tests and callers. It examines the commit timeline and discovers that the relevant behavior changed twice in the past year. It finds a recently modified function that sits on the same execution path. It might produce a structure like:

Issue → related historical PR → commit → function → caller → test

and separately:

Issue → lexical similarity → older issue

and:

recent commit → modifies function → test subsequently began failing.

None of those arrows requires an LLM.

At that point the agent gets a deliberately constructed investigation context containing those facts and relationships. Its job is to ask the next-order questions: "Could these two issues have the same root cause?" "Does the recent commit plausibly explain the regression?" "What historical design decision should the contributor know about?" "What experiment would

distinguish hypothesis A from hypothesis B?" That is where the model becomes enormously valuable, because these are abductive and conceptual reasoning tasks.

The semantic engine then becomes the adversarial counterpart to the agent. Suppose the agent says, "Function X was introduced by commit C." The engine checks Git and says yes or no. It says "this patch should fix issue 72"; the engine can't prove that, but it can check whether the patch modifies the execution path implicated by the issue. The agent says "there are no tests"; the engine can search its test graph and potentially refute that. The agent proposes a patch; the engine applies it in a worktree, calculates its semantic impact, runs relevant tests, and records the results.

This gives you a beautiful feedback cycle:

forge event → deterministic ingestion → relationship discovery → agent interpretation → hypothesis → deterministic experiment → new evidence → revised hypothesis → patch

And importantly, the entire chain can be persisted.

The persistence model I'd use is closer to an event-sourced knowledge base than a conventional cache. You retain raw observations and derived facts with their provenance. If your analysis algorithm improves six months later, you can recompute relationships from the underlying observations. You don't want the repository's "intelligence" to consist of irreversible LLM summaries that nobody can audit.

A Rust implementation could have something like an ingestion engine, Git index, source-language index, forge index, relationship engine, query engine, and local daemon. SQLite is probably enough for the first serious persistent store. You could store structured entities and relationships relationally while maintaining indexes for the common graph queries. You don't need Neo4j or some elaborate graph database initially. The graph is a conceptual model; relational storage can represent it perfectly well at this scale.

The protocol between that engine and Oculus could then be extremely simple. Oculus asks: "Give me the investigation context for issue 183." The engine returns structured evidence. Oculus displays it and sends a selected projection to the agent. The agent responds with hypotheses and requested investigations. Oculus sends those requests back to the engine. The engine executes deterministic queries and returns results. Eventually the UI becomes almost a REPL over the project's semantic model.

And that, I think, is the crucial answer to "how does deterministic parsing combine with agents?" The deterministic engine shouldn't try to become intelligent in the human sense, and the agent shouldn't be trusted to reconstruct objective repository facts. The engine constructs a progressively richer, queryable world model; the agent explores that model, proposes interpretations and hypotheses; the engine checks those proposals against reality; and the human chooses which line of inquiry is worth pursuing.

If you get that boundary right, you can make the agent dramatically more capable without making the system less trustworthy. The model gets to reason over relationships that would be prohibitively tedious to discover manually, while the user can always drill down from an interpretation to the exact Git commit, file, symbol, issue, test, or forge conversation that produced the underlying evidence. That's the foundation I'd build before attempting anything resembling autonomous repository intelligence.